Configure an Embedded Linux Device

Cross-compiling Qt for a given device requires a **toolchain** and a **sysroot**. The toolchain is expected to contain a version of gcc, or another compiler, and associated tools built for cross-compilation. This means these tools run on the **host** system (typically x64), while producing binaries for the target architecture (for example, 32 or 64 bit ARM). The sysroot contains the headers and libraries for the target system, allowing compiling and linking libraries and applications on the host.

This overview page describes to the generic approach, where no distribution building systems, such as Yocto or Buildroot, are used. It is always possible to cross-compile and deploy Qt onto a device as long as a suitable toolchain and sysroot are available.

> **Warning:** This page can only provide a generic, high-level overview. There are a vast number of details that can vary depending on the build environment, the target device, and the toolchain. When in doubt, refer to your system integrator. For pre-built reference images and SDKs, refer to the **Boot to Qt** offering.

When running Qt-based applications without a windowing system, such as X11 or Wayland, some devices require vendor-specific adaptation code for EGL and OpenGL ES support. This is provided in form of backends for the EGLFS platform plugin. This is not relevant for non-accelerated platforms, such as those that use the LinuxFB platform plugin, which is meant for software-based rendering only. As of Qt 6, many embedded systems use **drm** to set a video mode, manage display connectors and graphical surfaces. For example, an NXP i.MX8-based device or a Raspberry Pi 4 will use this approach, and therefore the most commonly used backend for EGLFS is **eglfs_kms**, which enables EGL and OpenGL ES based rendering with `drm`, using `gbm` for surface and buffer management. Older devices, such as the NXP i.MX6, will continue to use the legacy, GPU vendor-specific approach to connect EGL window surfaces to the framebuffer, using dedicated eglfs backends, such as `eglfs_viv`.

> **Note:** Be aware that Qt is just one component in the software stack for an embedded device. Especially when accelerated graphics are involved, Qt expects a functional graphics stack, with an appropriate configuration for the userspace and kernel components, such as the display driver. These components are outside of Qt's domain, and it is the system integrator's responsibility to ensure the base system is fully functional and optimal, including accelerated graphics.

For further information on graphics and input configuration for Embedded Linux systems, refer to **Qt for Embedded Linux**.

Toolchain Files versus Device Makespecs

In Qt 5, you would typically use a device spec under the *qtbase/mkspecs/devices* directory. These contain the appropriate compiler and linker flags for a certain device, also making sure the correct EGL and OpenGL ES libraries are picked up, in case they are in a non-standard location in the sysroot.

For example, you could have configured a Qt 5 build for a Raspberry Pi 2 with a configure command like the following:

```
./configure -release -opengl es2 -device linux-rasp-pi2-g++ -device-option CROSS_COMPILE=$TOOLCHAIN/arm-bcm2708/gcc-linaro-arm-linux-gnuea
```

> **Note:** configure always uses the **Ninja** generator and build tool if a `ninja` executable is available. Ninja is cross-platform, feature-rich, performant, and recommended on all platforms. The use of other generators might work but is not officially supported.

With Qt 6 and CMake, this approach is **no longer sufficient** on its own. Rather, a **CMake toolchain file** must be provided before configuring can happen. It is in this file where customization with regards to compiler and linker flags, and toolchain and sysroot specific quirks, happens.

In the below sections we will present a toolchain file that can be used in many cases, with minimal customization. It is based on the approach presented in **this blog post**.

> **Note:** The toolchain file presented below is an example, that will often need further customization for a given device. Users and system integrators are also free to create their own toolchain files in any way they see fit.

While CMake is the only supported build system for building Qt itself, applications may still be built using `qmake` in Qt 6.0. In order to get a `qmake` setup that is functional with cross-compilation, one will need to specify some of the legacy arguments to CMake or to configure.

Host Tools

Cross-compiling Qt requires a host build of Qt being available. See **Cross-compiling Qt** for details.

Configuring Qt

Let's assume that the following are available:

- a toolchain and sysroot under `$HOME/rpi-sdk`,
- a checkout of Qt, at minimum the qtbase module, under `$HOME/qt-cross`,
- a host build of Qt in `$HOME/qt-host`.

In addition, the following must be decided before configuring:

- Where will the Qt build be installed on the local system once the build completes? In the example we will use `$HOME/qt6_rpi`

In the example we are going to use a Raspberry Pi 4 SDK (toolchain+sysroot) generated via Yocto, but the instructions here are completely generic, with no dependency on Yocto. The steps are the same with any other toolchain and sysroot, once the toolchain file is updated with the correct cross compiler and other paths.

After creating and switching to a `build` directory:

```
$HOME/qt-cross/qtbase/configure -release -opengl es2 -nomake examples -nomake tests \
  -qt-host-path $HOME/qt-host                              \
  -extprefix $HOME/qt6-rpi                                 \
  -prefix /usr/local/qt6                                   \
  -- -DCMAKE_TOOLCHAIN_FILE=$HOME/qt-cross/toolchain.cmake
```

In practice this configure command is equivalent to the following direct CMake call:

```
cmake -GNinja -DCMAKE_BUILD_TYPE=Release -DINPUT_opengl=es2 -DQT_BUILD_EXAMPLES=OFF -DQT_BUILD_TESTS=OFF \
  -DQT_HOST_PATH=$HOME/qt-host                           \
  -DCMAKE_STAGING_PREFIX=$HOME/qt6-rpi                   \
  -DCMAKE_INSTALL_PREFIX=/usr/local/qt6                  \
  -DCMAKE_TOOLCHAIN_FILE=$HOME/qt-cross/toolchain.cmake  \
  $HOME/qt-cross/qtbase
```

Given the appropriate toolchain file, this is sufficient to generate a Qt build that then allows applications to be built using CMake. To enable applications to be built with `qmake` as well, the Qt 5 style device spec and device options must be specified, in addition to all arguments shown above:

```
$HOME/qt-cross/qtbase/configure ...
  ...
  -device linux-rasp-pi4-v3d-g++ \
  -device-option CROSS_COMPILE=$HOME/rpi_sdk/sysroots/x86_64-pokysdk-linux/usr/bin/arm-poky-linux-gnueabi/arm-poky-linux-gnueabi- \
  -device-option DISTRO_OPTS="hard-float" \
  ...
```

By default, when cross-compiling, only the Qt libraries and tools that are supposed to run on the target device are built. Build-related tools like `moc` and `uic` are not built. Building such tools can be enabled by setting `QT_FORCE_BUILD_TOOLS` to `ON`.

> **Note:** When `QT_FORCE_BUILD_TOOLS` is enabled, target binaries of tools like `qmake` will get installed to the staging location. Therefore, if `qmake` is used to build applications, call the `host-qmake` script instead.

Once configuration completes without errors, run `cmake --build . --parallel` to build. Once built, run `cmake --install .` to install the results to `$HOME/qt6-rpi`. From there the Qt build can be deployed to the device by using rsync, scp, or another method.

If building individual Qt modules, one can use the `qt-configure-module` script from the `bin` directory of the staging location (`$HOME/qt6-rpi` in the example) to configure additional modules, such as qtdeclarative, qtquick3d, and so on. They can then be built using `cmake --build .` and installed to the staging location by running `cmake --install .`

> **Note:** Before starting the build, always inspect the output of the configuration step carefully: does it have all the expected features enabled? Making a build and deploying it to the device is futile if essential features are not enabled at configuration time.

For example, when accelerated graphics via OpenGL is desired, pay extra attention to the following features:

```
EGL ................................... yes
OpenGL:
  Desktop OpenGL ...................... no
  OpenGL ES 2.0 ....................... yes
  OpenGL ES 3.0 ....................... yes
...
evdev ................................. yes
libinput .............................. yes
...
EGLFS ................................. yes
EGLFS details:
  EGLFS OpenWFD ....................... no
  EGLFS i.Mx6 ......................... no
  EGLFS i.Mx6 Wayland ................. no
```

```
    EGLFS VSP2 ........................... no
    EGLFS Mali ........................... no
    EGLFS Raspberry Pi ................... no
    EGLFS X11 ............................ no
  LinuxFB ................................ yes
```

With the Raspberry Pi 4 example, we expect that EGL, OpenGL ES and `EGLFS GBM` are all reported as `yes`, otherwise the EGLFS platform plugin and its **eglfs_kms** backend will not be functional on the device. For getting function mouse, keyboard, and touch input, either `evdev` or `libinput` must be enabled.

Similarly, if X11 is planned to be used as the (or one of the) windowing systems on the device, then ensure the xcb and X11 related features are marked as `yes`.

An Example Toolchain File

We will assume there is a sysroot and toolchain available under `$HOME/rpi-sdk`. `TARGET_SYSROOT` and `CROSS_COMPILER` must be adjusted to the toolchain and sysroot in use. The example here is only suitable for one specific, Yocto-generated SDK. The same is true for `CMAKE_C_COMPILER` and `CMAKE_CXX_COMPILER`.

We do not rely on any wrapper scripts that would provide environment variables such as PKG_CONFIG_*. Rather, the path to the .pc files is specified in the toolchain file. It is likely that another sysroot will need adjustments in `PKG_CONFIG_LIBDIR`. For example, with a sysroot generated from a Raspberry Pi OS (formerly Raspbian) image one would use `/usr/lib/arm-gnueabihf/pkgconfig` instead.

The compiler and linker flags are not necessary optimal in the example. Adjust them as necessary for the target device.

For further information on the CMake specifics in the example toolchain file, refer to **this blog post** and **the CMake documentation**.

```
cmake_minimum_required(VERSION 3.18)
include_guard(GLOBAL)

set(CMAKE_SYSTEM_NAME Linux)
set(CMAKE_SYSTEM_PROCESSOR arm)

set(TARGET_SYSROOT /home/user/rpi-sdk/sysroots/cortexa7t2hf-neon-vfpv4-poky-linux-gnueabi)
set(CROSS_COMPILER /home/user/rpi-sdk/sysroots/x86_64-pokysdk-linux/usr/bin/arm-poky-linux-gnueabi)

set(CMAKE_SYSROOT ${TARGET_SYSROOT})

set(ENV{PKG_CONFIG_PATH} "")
set(ENV{PKG_CONFIG_LIBDIR} ${CMAKE_SYSROOT}/usr/lib/pkgconfig:${CMAKE_SYSROOT}/usr/share/pkgconfig)
set(ENV{PKG_CONFIG_SYSROOT_DIR} ${CMAKE_SYSROOT})

set(CMAKE_C_COMPILER ${CROSS_COMPILER}/arm-poky-linux-gnueabi-gcc)
set(CMAKE_CXX_COMPILER ${CROSS_COMPILER}/arm-poky-linux-gnueabi-g++)

set(QT_COMPILER_FLAGS "-march=armv7-a -mfpu=neon -mfloat-abi=hard")
set(QT_COMPILER_FLAGS_RELEASE "-O2 -pipe")
set(QT_LINKER_FLAGS "-Wl,-O1 -Wl,--hash-style=gnu -Wl,--as-needed")

set(CMAKE_FIND_ROOT_PATH_MODE_PROGRAM NEVER)
set(CMAKE_FIND_ROOT_PATH_MODE_LIBRARY ONLY)
set(CMAKE_FIND_ROOT_PATH_MODE_INCLUDE ONLY)
set(CMAKE_FIND_ROOT_PATH_MODE_PACKAGE ONLY)

include(CMakeInitializeConfigs)

function(cmake_initialize_per_config_variable _PREFIX _DOCSTRING)
  if (_PREFIX MATCHES "CMAKE_(C|CXX|ASM)_FLAGS")
    set(CMAKE_${CMAKE_MATCH_1}_FLAGS_INIT "${QT_COMPILER_FLAGS}")

    foreach (config DEBUG RELEASE MINSIZEREL RELWITHDEBINFO)
      if (DEFINED QT_COMPILER_FLAGS_${config})
        set(CMAKE_${CMAKE_MATCH_1}_FLAGS_${config}_INIT "${QT_COMPILER_FLAGS_${config}}")
      endif()
    endforeach()
  endif()

  if (_PREFIX MATCHES "CMAKE_(SHARED|MODULE|EXE)_LINKER_FLAGS")
    foreach (config SHARED MODULE EXE)
      set(CMAKE_${config}_LINKER_FLAGS_INIT "${QT_LINKER_FLAGS}")
    endforeach()
  endif()
```

Building Applications for the Target Device

Once the Qt build is done and installed to the staging location, examples or applications can be built.

With CMake, use the generated `qt-cmake` script in the `bin` directory of the staging location (`$HOME/qt6-rpi` in the example) to configure, then run `ninja`. For example:

```
$HOME/qt6-rpi/bin/qt-cmake .
cmake --build .
```

The resulting application binary can then be deployed to the device. Using the `qt-cmake` helper script is convenient, because the script ensures the toolchain file that was used for building Qt is loaded, so there is no need to repeatedly specify it for each application.

Unlike for Qt itself, building applications with qmake is still supported in Qt 6.0, as long as a suitable device spec is available, and the appropriate legacy arguments were passed to CMake or configure when configuring Qt. If this is all true, then running `qmake` and `make` will also generate an application binary for the target device.

Defaults for Platform Plugins and EGLFS

Once configured, a default platform plugin is chosen. This is used when launching an application without the `-platform` argument and without having the `QT_QPA_PLATFORM` environment variable set.

Similarly, the EGLFS platform plugin has multiple backends. The default is chosen based on availability and a pre-defined priority order. If drm and gbm are available, the default will be the **eglfs_kms** backend. This can always be overridden at runtime by setting the `QT_QPA_EGLFS_INTEGRATION` environment variable.

To change these defaults for the build, without having to force a specific value at run time, the following two CMake cache variables are available after CMake has been run once:

- `QT_QPA_DEFAULT_PLATFORM` (`STRING`) - The name of the default platform plugin.
- `QT_QPA_DEFAULT_EGLFS_INTEGRATION` (`STRING`) - The default EGLFS backend.

These variables can also be set inside the toolchain file.

For more information on configuring Qt, see Qt Configure Options.

‹ Qt for Embedded Linux

Inputs on an Embedded Linux Device ›

© 2026 The Qt Company Ltd. Documentation contributions included herein are the copyrights of their respective owners. The documentation provided herein is licensed under the terms of the GNU Free Documentation License version 1.3 as published by the Free Software Foundation. Qt and respective logos are trademarks of The Qt Company Ltd. in Finland and/or other countries worldwide. All other trademarks are property of their respective owners.

QtWS26 The Qt Event of the Year: Qt World Summit 2026 Save Your Spot ›

Contact Us
Qt Group
Our Story
Brand
News
Careers
Investors
Qt Products
Software Quality Products
Licensing
License Agreement
Open Source
Plans and pricing
Download
FAQ
Learn Qt
For Learners
For Students and Teachers
Qt Documentation
Qt Forum
Support & Services
Professional Services
Customer Success
Support Services
Partners
Qt World
© 2026 The Qt Company
Feedback
Qt Group includes The Qt Company Oy and its global subsidiaries and affiliates.